LD10 Runtime-Derived Density Resource Policy Specification

Host-aware memory, thread, wall-time, worker, and browser-output budgeting

mdstats development specification

2026-07-21

Contents

Status and scope

This specification defines **LD10** for `mdstats 0.19.64a0`. It replaces trajectory- or benchmark-fitted density compute caps with one runtime-derived resource policy for framework-dynamics density preparation and rendering.

The policy covers:

- runtime CPU and memory discovery;
- the default 80% memory and 90% CPU fractions;
- the default 20-minute complete-scene wall-time objective;
- explicit API, command-line, and environment overrides;
- scene-wide peak-memory and wall-time admission;

- low-level density planning, sparse realization, dense realization, contouring, caches, and mesh-worker limits;
- child-process propagation and native-library thread containment;
- strict separation of host-compute limits from browser-output profiles.

It does **not** change density estimators, registration, grid resolution, Gaussian bandwidth, HDR semantics, mesh geometry, scientific tolerances, or browser fidelity criteria.

Engineering rule

The normative rule is:

> A package compute limit must be derived from the current runtime allocation and the requested wall-time objective. It must not be increased because one example system nearly exceeds a previous cap.

Historical `DEFAULT_MAX_*` compute constants remain importable only for compatibility. They are not active admission defaults. A static regression test rejects future use of those constants as compute defaults.

Three budget domains

Resource policy has three non-interchangeable domains.

Host compute budget

The host budget controls package-owned additional memory, native/process threads, and complete-scene wall time:

```
FrameworkDynamicsResources(
    max_memory_bytes=None,
    max_threads=None,
    max_wall_time_seconds=None,
    memory_fraction=0.80,
    thread_fraction=0.90,
)
```

These values govern scientific planning and execution.

Algorithmic work estimates

Exact or conservative scene counts are compared with the host budget:

- retained and transient bytes;
- trajectory points and sample arrays;
- dense nodes, sparse nodes, block slots, and planning arrays;
- CIC insertions, stencil values, and kernel pairs;
- mesh cells, raw faces, raw vertices, and worker workspaces;
- calibrated preparation and rendering wall time.

The counts describe the requested input. The limits against which they are checked are runtime-derived and therefore input-independent.

Browser-output profile

Final Plotly faces, vertices, traces, and HTML bytes are client-delivery constraints. They remain explicit browser profiles such as `BrowserMeshBudget`. They do not enlarge or reduce host compute admission and are not inferred from system RAM.

Runtime discovery

CPU allocation

Let

- T_{aff} be the current process CPU-affinity count;
- T_{cgroup} be the floor of the cgroup CPU quota when finite;
- T_{sched} be the smallest applicable scheduler CPU declaration.

The available runtime CPU count is

$$T_{\text{available}} = \max\left(1, \min\left(T_{\text{aff}}, T_{\text{cgroup}}, T_{\text{sched}}\right)\right),$$

where absent terms are omitted. The scheduler candidates are `SLURM_CPUS_PER_TASK`, `SLURM_CPUS_ON_NODE`, `PBS_NP`, `NSLOTS`, and `LSB_DJOB_NUMPROC`.

The default thread budget is

$$T_{\text{default}} = \max\left(1, \lfloor 0.8 T_{\text{available}} \rfloor\right).$$

CPU affinity is obtained through `os.sched_getaffinity` where available. Linux cgroup v2 `cpu.max` and cgroup v1 quota files are treated as upper bounds. Scheduler variables are also upper bounds, not proof that all CPUs are idle.

Memory headroom

Let the candidate additional-memory headrooms be:

- M_{host}: `/proc/meminfo` `MemAvailable`, or an operating-system available-memory fallback;
- $M_{\text{cgroup}} = M_{\max} - M_{\text{current}}$ for a finite cgroup limit;
- $M_{\text{sched}} = M_{\text{allocation}} - M_{\text{RSS}}$ for a scheduler declaration;
- $M_{\text{rlimit}} = M_{\text{RLIMIT\_AS}} - M_{\text{virtual}}$ for a finite address-space limit.

The detected additional-memory ceiling is

$$M_{\text{available}} = \max\left(1, \min\left(M_{\text{host}}, M_{\text{cgroup}}, M_{\text{sched}}, M_{\text{rlimit}}\right)\right),$$

with absent terms omitted. A zero authoritative headroom is retained as zero before the final one-byte representational floor; it is never discarded in favor of a larger host value.

When multiple scheduler memory variables are present, the most restrictive parsable value is used. Supported declarations include `SLURM_MEM_PER_NODE`, `SLURM_MEM_PER_CPU`, `PBS_RESC_MEM`, `PBS_VMEM`, and `LSB_MAX_MEM`.

The default package budget is

$$M_{\text{default}} = \max\left(1, \lfloor 0.8 M_{\text{available}} \rfloor\right).$$

The remaining 20% is deliberate operating-system, allocator, interpreter, input-data, and third-party-library headroom. An explicit user value may use more, but it is clamped to the detected runtime ceiling.

Wall-time objective

The default complete-scene objective is

$$W_{\text{default}} = 1200\ \text{s}.$$

The timer covers scientific scene preparation and density rendering. Calibration time is recorded separately and is designed to be negligible. Isolated mesh workers receive a timeout no larger than the remaining scene wall time. Main-process numerical kernels are admitted by conservative preflight and checked at stage boundaries; Python cannot reliably preempt every native kernel without risking corrupted state.

User overrides

Python API

```
resources = FrameworkDynamicsResources(
    max_memory_bytes="12GiB",
    max_threads=16,
    max_wall_time_seconds=1800,
)
```

`max_memory_bytes` accepts an integer byte count or `KB`, `MB`, `GB`, `TB`, `KiB`, `MiB`, `GiB`, or `TiB`. Explicit memory and thread values are clamped to the detected runtime allocation. An explicit wall time may be larger or smaller than 20 minutes.

Legacy low-level count arguments may only tighten the resolved budget. They cannot be used to increase memory, operation, cache, block, node, pair, or mesh limits beyond the active scene budget.

Environment

The equivalent process-level controls are:

```
MDSTATS_MAX_MEMORY_BYTES
MDSTATS_MAX_THREADS
MDSTATS_MAX_WALL_TIME_SECONDS
```

Precedence is:

$$\text{explicit API argument} > \text{MDSTATS environment override} > \text{runtime-derived default}\ .$$

Every resolved record stores the source and whether clamping occurred.

Example command line

The all-species framework-density example exposes:

```
--max-memory
--max-threads
--max-wall-time
--max-browser-faces
```

The first three are host-compute controls. `--max-browser-faces` is intentionally a separate output control.

Authoritative scene budget

`RuntimeResourceBudget` is immutable and contains:

```
max_memory_bytes: int
max_threads: int
max_wall_time_seconds: float
memory_fraction: float
thread_fraction: float
snapshot: RuntimeResourceSnapshot
override provenance and clamp flags
```

`density_resource_budget_scope(budget)` stores the complete-scene budget in a `ContextVar`. Every nested planner and kernel inherits the exact scene values. This prevents:

- a second 80% reduction in a low-level helper;
- independent re-probes producing inconsistent limits within one transaction;
- serialized low-level limits bypassing a smaller current runtime;
- concurrent tasks overwriting one another's budgets.

An explicit nested value is resolved as

$$B_{\text{nested}} = \min(B_{\text{requested}}, B_{\text{scene}}).$$

Runtime-derived numeric guardrails

Broad guardrails are generated by `derive_density_numeric_limits`. Retained-array limits are memory-derived. Operation-only limits are time-derived. For a retained item with conservative size b_i,

$$N_{i,\text{memory}} = \left\lfloor \frac{M_{\max}}{b_i} \right\rfloor.$$

For an operation with calibrated conservative throughput r_i and time-model safety factor s,

$$N_{i,\text{time}} = \left\lfloor \frac{W_{\max}}{s} r_i \right\rfloor.$$

These individual values are coarse early rejection guards. They do not replace exact aggregate memory or summed wall-time planning.

Time model

Calibration

`DensityTimeModel` is calibrated from small synthetic, input-independent kernels:

- fractional-coordinate transforms;
- indexed accumulation with `numpy.bincount`;
- Gaussian exponential evaluation;
- forward/inverse FFT;
- scalar crossing-cell scans;
- `skimage.measure.marching_cubes` when available.

Calibration is bounded by the active thread budget using `threadpoolctl`. Standalone calibration requests are clamped to the current runtime CPU allocation. Median throughput is used rather than the fastest repetition. Only a conservative fraction of measured throughput enters admission, followed by a safety multiplier.

No atom count, cell shape, trajectory length, species, Na-LTA benchmark, or previous successful scene enters the calibration.

Preparation estimate

For F fields, N_s samples, N_k stencil values, N_p kernel pairs, and N_d dense nodes,

$$\hat{t}_{\mathrm{prep}} = s\left[Ft_F + \frac{N_s}{r_s} + \frac{N_k}{r_k} + \frac{N_p}{r_p} + \frac{N_d}{r_d}\right].$$

The complete scene is rejected before allocation when this estimate exceeds the available wall-time budget.

Mesh estimate

For S shells, N_c crossing/mesh cells, N_f raw faces, P workers, parallel efficiency η, and process-start cost t_0,

$$\hat{t}_{\mathrm{mesh}} = s\left[\frac{St_S + N_c/r_c + N_f/r_f}{\max(1, P\eta)} + \frac{St_0}{P}\right].$$

Serial and isolated-worker shell groups are estimated separately and added.

Memory accounting

Preparation transaction

For Phase-B planning bytes P, retained field bytes R_i, and transient construction bytes T_i in deterministic field order,

$$M_{\mathrm{prep,peak}} = \max\left[P + \sum_i R_i, \max_i\left(P + \sum_{j<i} R_j + T_i\right)\right].$$

The scene is rejected when

$$M_{\mathrm{prep,peak}} > M_{\max}.$$

Individual count caps cannot authorize a scene that fails this aggregate check.

Rendering transaction

Let

- M_{parent} be recursively retained NumPy buffers owned by the prepared scene;
- M_{output} be reserved final geometry/serialization bytes;
- M_{serial} be the largest parent-process contour workspace;

- M_{worker} be the largest isolated worker peak.

The serial bound is

$$M_{\text{serial,peak}} = M_{\text{parent}} + M_{\text{output}} + M_{\text{serial}}.$$

For P isolated workers,

$$M_{\text{pool,peak}} = M_{\text{parent}} + M_{\text{output}} + PM_{\text{worker}}.$$

Both must be no larger than M_{max}.

Worker allocation

For native threads per worker t_w, isolated shell count S, and memory available to the worker pool

$$M_{\text{pool}} = M_{\text{max}} - M_{\text{parent}} - M_{\text{output}},$$

the default worker count is

$$P = \min\left[S, \left\lfloor \frac{T_{\text{max}}}{t_w} \right\rfloor, \left\lfloor \frac{M_{\text{pool}}}{M_{\text{worker}}} \right\rfloor\right].$$

The default is one native numerical thread per isolated worker. A user may request another value, but the total remains within the scene thread budget. Each child process receives explicit values for:

```
OMP_NUM_THREADS
OPENBLAS_NUM_THREADS
MKL_NUM_THREADS
BLIS_NUM_THREADS
VECLIB_MAXIMUM_THREADS
NUMEXPR_NUM_THREADS
MDSTATS_MAX_THREADS
MDSTATS_MAX_MEMORY_BYTES
MDSTATS_MAX_WALL_TIME_SECONDS
```

This prevents nested BLAS/OpenMP oversubscription and prevents child low-level helpers from seeing the full parent allocation.

Cache policy

Stencil and routing caches are process-local, byte-bounded, clearable LRU caches. At use time their entry counts and retained bytes are derived from the active runtime budget. Historical constructor constants initialize empty caches only; they do not admit any allocation. A cached object is revalidated against the current caller's limits before reuse.

Failure policy

Resource failure must be explicit and pre-allocation whenever the required count is known. Errors report:

- requested/estimated count or bytes;
- resolved runtime limit;
- relevant retained, transient, worker, and output terms;
- remediation through explicit memory/thread/wall-time controls or reduced scientific resolution/request size.

The implementation must not silently:

- change grid interval or Gaussian bandwidth solely to fit resources;
- omit species, frames, density fields, or HDR shells;
- switch from mesh to point rendering;
- raise an internal count cap until a benchmark passes;
- treat a browser face budget as additional host memory.

API and serialization

The public runtime records are:

```
RuntimeResourceSnapshot
RuntimeResourceBudget
DensityTimeModel
FrameworkDynamicsResources
DensityPlanningLimits
DensityMeshExecutionOptions
```

All records are schema-versioned and JSON-serializable. Deserialization re-applies the current active budget, so a record created on a larger host cannot relax limits on a smaller host.

`runtime_snapshot` and direct `time_model` injection exist for deterministic testing and expert diagnostics. They are not command-line controls and are not a substitute for `max_memory_bytes`, `max_threads`, or `max_wall_time_seconds` in production.

Edge cases

1. **One available CPU.** The default remains one thread.
2. **Less than one byte reported headroom.** The snapshot records a one-byte representational floor and all useful allocations fail preflight.
3. **Unlimited cgroup.** The cgroup term is omitted; affinity, host, scheduler, and process limits still apply.
4. **Fractional CPU quota.** The quota is floored, with a minimum of one CPU.
5. **Several scheduler memory variables.** The smallest parsable allocation is used.
6. **Malformed scheduler variable.** It is ignored rather than converted to a tiny accidental cap; explicit `MDSTATS_*` overrides fail validation instead.
7. **Scene created after memory pressure changes.** A new `FrameworkDynamicsResources` object performs a fresh probe. The package does not globally cache memory headroom.
8. **Concurrent scenes.** `ContextVar` scoping keeps budgets independent, but total shared-host pressure remains external; the host/cgroup headroom probe supplies the available upper bound at each scene's start.
9. **No `threadpoolctl`.** Complete framework-dynamics plotting rejects execution because native-thread containment cannot be guaranteed.
10. **Calibration unavailable.** A conservative static fallback is used and its source is recorded.
11. **Native kernel exceeds estimate.** Stage-boundary elapsed checks report the overrun; isolated workers are also subject to subprocess timeouts.
12. **Broad sparse field.** Backend selection may choose dense storage based on exact plans; resource limits do not force sparse storage.

Validation requirements

The focused suite must verify:

- 80% default memory and 90% default CPU fractions;
- 20-minute default wall time;
- API and environment override precedence;
- clamping to CPU and memory runtime ceilings;
- fresh memory probes across calls;
- the most restrictive simultaneous scheduler memory declaration;
- runtime-derived count scaling independent of any atomistic fixture;
- operation-count scaling with wall time;
- exact active-budget inheritance without double fractioning;
- nested tightening-only behavior and context restoration;
- serialized/legacy low-level controls cannot relax the active budget;
- calibration cannot oversubscribe available CPUs;
- worker counts are bounded jointly by CPU, memory, and shell count;
- worker native threads and timeouts are bounded by the scene;
- child environment propagation;

- browser profiles remain separate from host-compute policy;
- static rejection of active historical fixed compute caps;
- all-species example code contains no fixed host-compute caps.

Known limitations

- Memory estimates cover package-owned arrays and declared third-party workspaces, not every allocator-internal page, JIT cache, driver buffer, or unrelated thread/process.
- Scheduler memory variables may describe a job shared by sibling processes. Cgroups are preferred when available because their current usage includes the group.
- Throughput calibration is deliberately conservative but cannot guarantee an exact deadline on every NUMA topology, storage condition, thermal state, or competing workload.
- Main-process native kernels are not forcibly interrupted mid-call.
- Browser budgets still require explicit client-profile validation; system RAM does not predict WebGL capability.

References

1. Linux kernel documentation, *Control Group v2*, especially `cpu.max`, `memory.max`, and `memory.current`: https://docs.kernel.org/admin-guide/cgroup-v2.html.
2. Linux kernel documentation, *Memory Resource Controller* (cgroup v1): https://docs.kernel.org/admin-guide/cgroup-v1/memory.html.
3. Python documentation, `os.sched_getaffinity`: https://docs.python.org/3/library/os.html#os.sched_getaffinity.
4. Python documentation, `resource.RLIMIT_AS`: https://docs.python.org/3/library/resource.html#resource.RLIMIT_AS.
5. SchedMD, Slurm `srun` output environment variables, including `SLURM_CPUS_ON_NODE` and `SLURM_CPUS_PER_TASK`: https://slurm.schedmd.com/srun.html#SECTION_OUTPUT-ENVIRONMENT-VARIABLES.
6. `threadpoolctl`, native BLAS/OpenMP thread-pool containment: https://github.com/joblib/threadpoolctl.